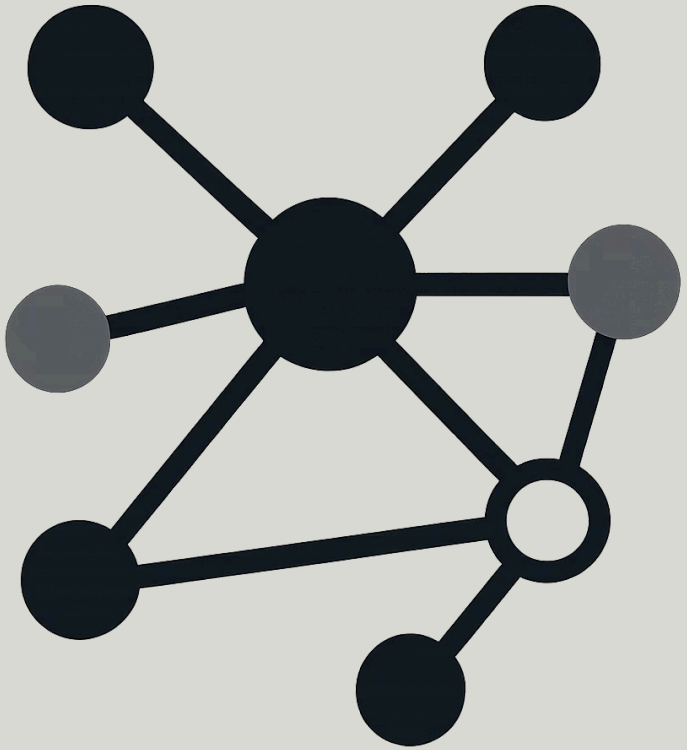
NAMING IN DISTRIBUTED SYSTEMS

Understanding how systems uniquely identify and locate entities across networks

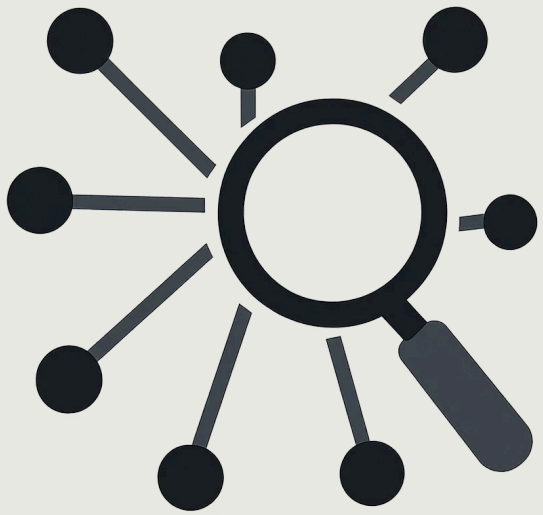


Made with GAMMA

WHAT IS NAME RESOLUTION?

Names are fundamental to all computer systems, enabling resource sharing and unique entity identification. The core functionality of any naming system is **name resolution**—the process of resolving a name into the specific entity it refers to.

In distributed systems, the naming system itself spans multiple machines, and how this distribution is managed dictates system efficiency and scalability.



Made with GAMMA

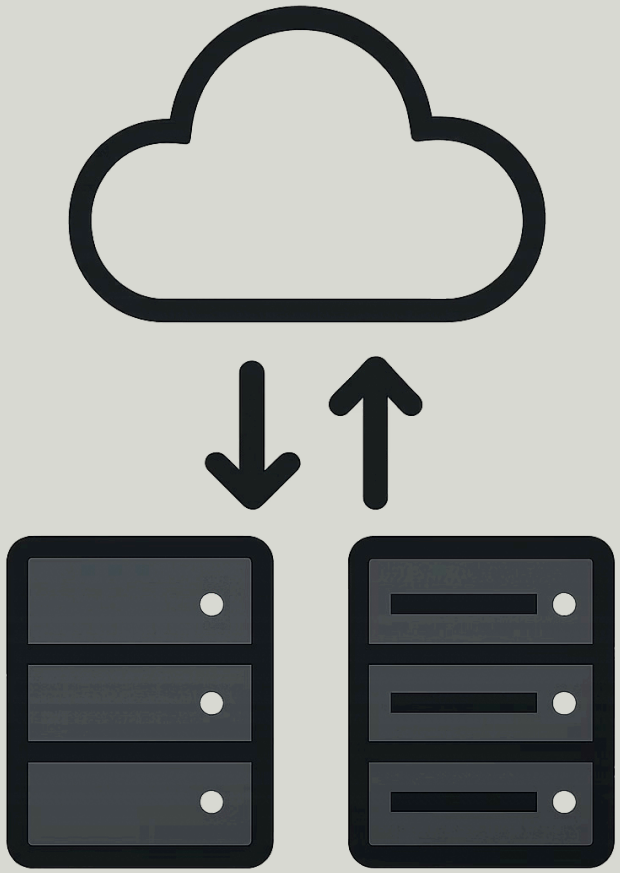
LOCATION INDEPENDENCE

DECOUPLING IDENTITY FROM ADDRESS

Names detach logical identity from physical network address, allowing references to remain valid even when entities move.

TRUE IDENTIFIERS

Unambiguous identifiers that refer to at most one entity, each entity has one identifier, and identifiers are never reused.



Made with GAMMA

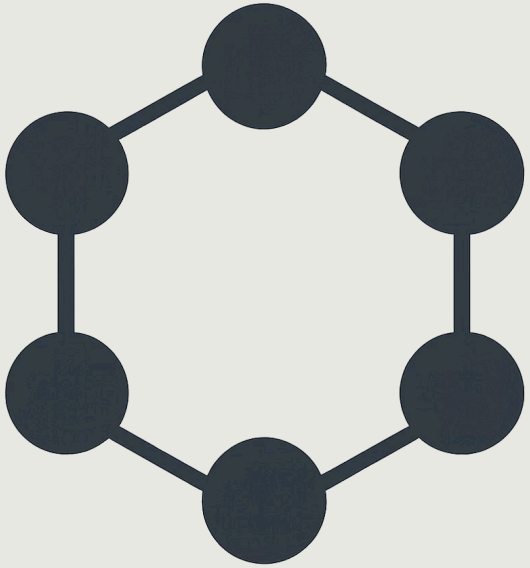
FLAT NAMING

STRUCTURELESS IDENTIFIERS

Entities are referred to by identifiers with no inherent meaning or structure. Special routing mechanisms are required to trace entity locations.

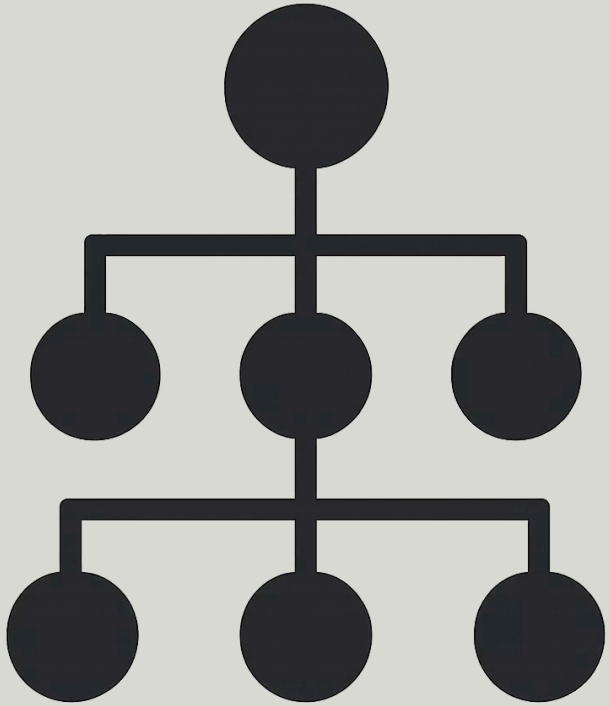
COMMON MECHANISMS

- Broadcasting
- Chains of forwarding links
- Home-based approaches
- Distributed Hash Tables (DHTs)



Chord protocol uses flat naming to route lookup requests around a logical ring of nodes until finding the requested data.

Made with GAMMA



STRUCTURED NAMING

HIERARCHICAL ORGANIZATION

Names organized into a tree structure with directory nodes and leaf nodes, using readable path names.

PATH RESOLUTION

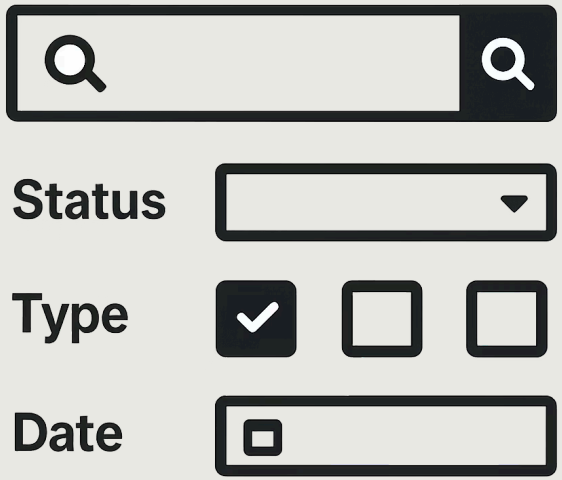
Resolution steps through label sequences to find the final entity, like navigating folders in a file system.

GLOBAL DISTRIBUTION

Domain Name System (DNS) distributes hierarchical trees globally, resolving human-readable URLs to IP addresses.

Made with GAMMA

ATTRIBUTE-BASED NAMING



DESCRIPTIVE LOOKUP

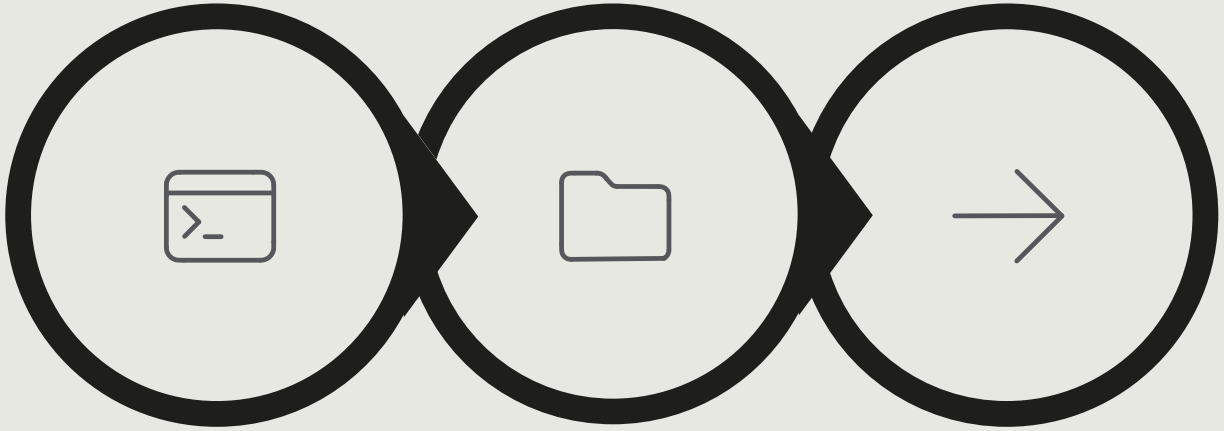
Resources are looked up by evaluating (attribute, value) pairs rather than single distinct names.

KEY TECHNOLOGIES

- Resource Description Framework (RDF) uses triplets: subject, predicate, object
- Lightweight Directory Access Protocol (LDAP) organizes attributes into Directory Information Base
- Widely used in enterprise directory services like Active Directory

Made with GAMMA

HOW NAME RESOLUTION WORKS



SUBMIT PATH

LOOKUP LABEL

STEP DOWN

Made with GAMMA

KEY ROLES IN DISTRIBUTED SYSTEMS

TRACKING MOBILE ENTITIES

Home-based approaches and forwarding pointers allow entities to change network location while keeping references valid.

SECURITY THROUGH NAMING

Self-certifying names hash immutable data objects, guaranteeing unique and secure binding between entity and name.

RESOURCE ABSTRACTION

Provides abstraction over network layer, preventing clients from needing to know specific server addresses.

Made with GAMMA

ADVANTAGES VS. CHALLENGES

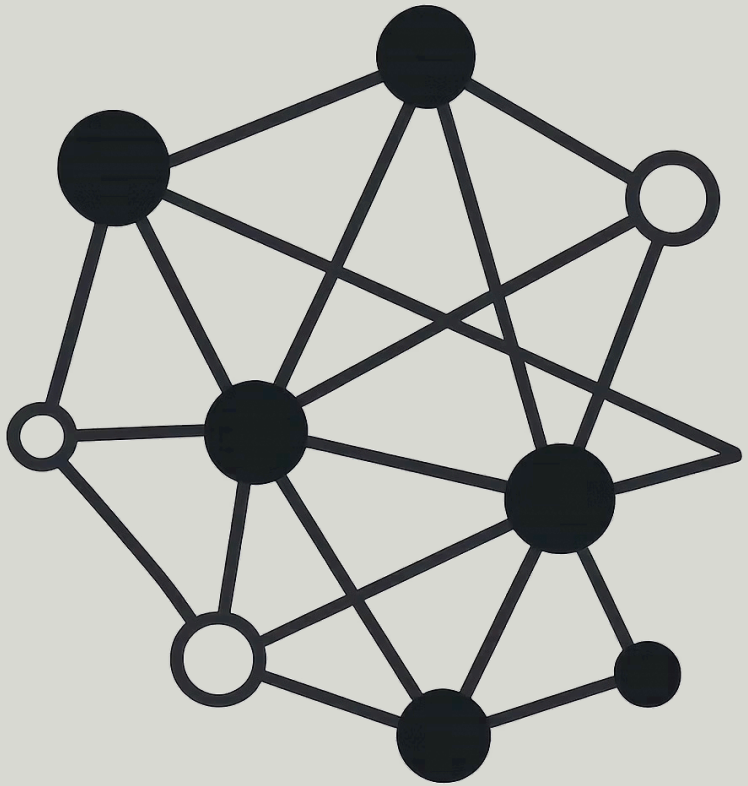
ADVANTAGES

- **Flexibility:** Location-independent names allow address reassignment without breaking references
- **Scalability:** Distributed implementation manages millions of globally distributed entities
- **User-Friendly:** Human-readable text instead of raw IP addresses

CHALLENGES

- **Resolution Overhead:** Multiple server communications introduce latency
- **Maintenance Complexity:** Managing distributed systems requires reducing pointer chains
- **Search Difficulty:** Querying decentralized systems requires complex distributed indexes

Made with GAMMA



SUMMARY

ESSENTIAL MECHANISM

Uniquely identifies entities, shares resources, and refers to physical locations in distributed computing.

THREE APPROACHES

Flat naming for peer-to-peer networks, structured naming for human-friendly resolution like DNS, and attribute-based naming for characteristic searches.

MODERN NETWORKS

Decoupling logical names from physical addresses provides flexibility, mobility, and scalability despite resolution complexities.

Made with GAMMA

Naming in a Distributed System

1. Introduction

Names play a crucial role in all computer systems because they are used to share resources, uniquely identify entities, and refer to locations. The core functionality of any naming system is **name resolution**, which allows a process to resolve a name into the specific entity it refers to, thus granting the process access to that named entity.

The primary difference between naming in a centralized system versus a distributed system lies in implementation: in a distributed system, the naming system itself is often distributed across multiple machines. How this distribution is managed dictates the efficiency and scalability of the entire system.

2. Use of Naming in the Distributed System

In a distributed system, a name is essentially a string of bits or characters used to refer to an entity. Naming is primarily used to detach the logical identity of a resource from its physical network address.

- **Location Independence:** If an address is used directly to refer to an entity, the reference immediately becomes invalid the instant the entity moves or the access point changes. Therefore, systems use a separate name that is independent of the server's address, resulting in a location-independent name.
- **True Identifiers:** Names are also used to unambiguously identify entities. A "true identifier" has strict properties: it refers to at most one entity, each entity is referred to by at most one identifier, and an identifier is never reused.

3. Types of Naming

Names in distributed systems are generally categorized into three different approaches:

A. Flat Naming

- **Concept:** In flat naming, entities are referred to by an identifier that fundamentally has no meaning and bears no structure. Because flat names lack structure, special routing mechanisms are required to trace the location of the entity.
- **Examples:** Mechanisms include broadcasting, chains of forwarding links, home-based approaches, and Distributed Hash Tables (DHTs).

- **Real Use:** Flat naming is extensively used in peer-to-peer networks via the Chord protocol. In Chord, a flat identifier is used to mathematically route a lookup request around a logical ring of nodes until it finds the specific machine holding the requested data.

B. Structured Naming

- **Concept:** Humans prefer to use readable, structured names. Structured naming systems organize names into a hierarchical naming graph consisting of directory nodes and leaf nodes.
- **Examples:** Path names like N:[label1, label2, ..., labeln], where name resolution steps through the sequence of labels to find the final entity.
- **Real Use:** The Domain Name System (DNS) is the most famous example of a structured naming system. DNS distributes its hierarchical tree globally, allowing users to look up human-readable URLs (like cs.vu.nl) and resolve them to specific IP addresses.

C. Attribute-Based Naming

- **Concept:** Humans often prefer to describe entities by means of various characteristics or attributes. In this approach, a resource is looked up by evaluating a list of (attribute, value) pairs rather than a single distinct name.
- **Examples:** The Resource Description Framework (RDF) describes resources as triplets consisting of a subject, a predicate, and an object (e.g., *Person, name, Alice*).
- **Real Use:** The Lightweight Directory Access Protocol (LDAP) organizes attributes into a Directory Information Base, widely used in enterprise directory services like Microsoft's Active Directory to query users, network printers, and file shares based on their properties.

4. Importance of Naming in the Distributed System

Naming is fundamental because it provides an abstraction over the underlying network layer. If a web service is distributed across several servers, using a single location-independent name prevents clients from needing to know which specific server address is the best one to use. Naming systems ensure that processes can verify if they are accessing the same entity by simply comparing true identifiers. Without naming, sharing resources and managing mobile or replicated entities across a distributed architecture would be practically impossible.

5. How Naming Processes Operate at the Heart of Distributed Systems

At the heart of a distributed system, processes interact heavily with naming graphs. A process submits a path name to the naming system, initiating a lookup called name resolution.

- The process starts at a logical node (like the root) and looks up the first label in the node's directory table.
- The system returns the identifier of the next node, and the resolution continues stepping down the path.
- In distributed systems, this process can be *iterative* (the client process queries each intermediate server one by one) or *recursive* (the server handles the subsequent lookups and returns the final answer).

6. The Role of Naming in Distributed Systems

The primary role of naming is to facilitate tracking, locating, and securing entities.

- **Tracking Mobile Entities:** By using home-based approaches or forwarding pointers, naming systems allow entities (like mobile devices or migrating processes) to change their network location while keeping their reference valid.
- **Ensuring Security:** Naming plays a role in security through "self-certifying names." By hashing an immutable data object and using that hash value as its name, a system guarantees that the entity and its name are uniquely and securely bound to each other.

7. Advantages and Disadvantages of Naming in Distributed Systems

Advantages:

- **Flexibility and Mobility:** Location-independent names allow the system to reassign addresses or migrate services without breaking client references.
- **Scalability:** By distributing the naming implementation (like the hierarchical structure of DNS or the finger tables in DHTs), naming systems can scale to manage millions of globally distributed entities.

- **User-Friendliness:** Structured and attribute-based naming allows human users to easily discover and query resources using understandable text and descriptors instead of raw IP addresses.

Disadvantages:

- **Resolution Overhead:** Looking up a name in a distributed system requires communicating with multiple intermediate servers across the network, which introduces latency compared to resolving a name locally.
- **Maintenance Complexity:** Managing distributed naming systems is difficult. For example, in flat naming using forwarding pointers, chains of pointers must be periodically reduced to prevent them from becoming too long and fragile.
- **Difficulty in Search:** Designing an appropriate set of attributes for attribute-based naming is notoriously difficult. Furthermore, querying a decentralized peer-to-peer system for specific attributes is largely infeasible without designing complex distributed mapping indexes.

8. Summary

Naming is an essential mechanism in distributed computing used to uniquely identify entities, share resources, and refer to physical locations. Systems utilize **flat naming** for raw, structure-less identification heavily used in peer-to-peer networks; **structured naming** to provide human-friendly hierarchical resolution like DNS ; and **attribute-based naming** to allow users to search for entities based on specific characteristics. By decoupling the logical name from the physical address, distributed naming systems provide the flexibility, mobility, and scalability required to operate modern networks, despite introducing complexities in name resolution and directory maintenance.

Message-oriented communication

1. How Processes Communicate (The Heart of Distributed Systems)

Processes play a fundamental role in distributed systems, serving as the foundational basis for communication between different machines. Because distributed systems lack a global shared memory, all communication inherently relies on sending and receiving low-level messages across a network.

To hide the extreme complexities of network mechanics and varied hardware, process communication is typically structured using layered protocols and heavily relies on **middleware**. Middleware sits above the standard operating system network layers and provides high-level, application-independent communication services. By using middleware constructs, a process can execute commands or transfer data to remote machines while maintaining **distribution transparency**—the illusion that the distant process is acting locally.

2. Introduction: Why Messaging?

Message-oriented communication addresses the fundamental limitations of traditional distributed communication paradigms like Remote Procedure Calls (RPC) and Remote Method Invocations (RMI). RPC and RMI enforce a tightly coupled model where both the caller and the callee must be up and running simultaneously at the exact time of communication. Additionally, these models require both parties to know exactly how to refer to one another.

This strict, tight coupling is often a serious drawback in distributed systems, as it limits flexibility, fault tolerance, and scalability. To overcome these challenges, message-oriented middleware (MOM) was developed to allow applications to exchange information independently through discrete messages.

3. Various Forms of Communication and Their Issues

Because different applications have varying requirements, distributed systems rely on several specialized forms of communication.

3.1 Message-Oriented Communication

This form focuses on the discrete transfer of data through messages.

- **Transient Messaging:** Technologies like Sockets or the Message-Passing Interface (MPI) facilitate direct, high-speed communication.

    - *Issues:* Transient communication assumes that serious failures are fatal. If a recipient process is offline or fails, the messages are typically lost.

- **Persistent Messaging:** Message-Oriented Middleware (MOM), such as AMQP or IBM WebSphere MQ, provides intermediate-term storage using queues.
    - *Issues:* While robust, persistent messaging introduces complex design challenges, such as how to address and map logical names to the correct destination queues across different networks.

3.2 Stream-Oriented Communication

Unlike discrete messages, stream-oriented communication is designed for a continuous data flow. It is widely used to transfer real-time data, such as audio and video.

- *Issues:* The primary challenge is timing and synchronization. If a network cannot guarantee steady bandwidth (Quality of Service), the streams will suffer from jitter and delay, ruining the real-time playback experience.

3.3 Multicast Communication

Multicasting involves a single sender simultaneously disseminating data to multiple receivers. Modern distributed systems often achieve this using application-level overlay networks rather than hardware routers.

- *Issues:* * **Routing Inefficiencies:** Multicast trees often suffer from *link stress* (a packet needlessly crossing the same physical link multiple times) and *stretch* (a penalty in delivery delay).
    - **Fault Tolerance:** Ensuring a message reaches *all* group members even when processes crash requires complex implementations like virtual synchrony and atomic multicasting.

4. Advanced Message-Queuing Systems (MQS)

Persistent communication relies on Message-Queuing Systems to enable asynchronous, loosely coupled communication.

A. Basic MQS Architecture & Routers

In a standard architecture, a sender uses a Put primitive to append a message to its local queue. The local OS resolves the queue-level address into a transport-level address and sends it across the network. Because direct connections aren't always optimal, large systems use **Routers**—

intermediate queues that receive messages and forward them along the best path to the final destination queue.

B. The Role of a Message Broker

Because different applications often require entirely different messaging formats, systems employ a **Message Broker**. The broker acts as an application-level gateway. It consults a repository of conversion rules, translates the incoming message into the format expected by the destination client, and places the newly formatted message into the target queue.

C. IBM's WebSphere MQ Example

WebSphere MQ utilizes Queue Managers to govern local queues. To transfer a message to a remote queue, the system uses a **Message Channel Agent (MCA)**, which wraps the message into transport-level packets, transmits it over a logical channel, and unwraps it at the destination.

5. Stream Synchronization and Quality of Service

A. Quality of Service (QoS)

Continuous media requires guaranteed bounded minimum and maximum end-to-end delays. Applications create a QoS contract with the system. Using algorithms like the **Token Bucket**, the system regulates transmission speed over the network. Data packets must "grab a token" generated at a constant rate to be sent, preventing network congestion bursts and ensuring steady flow.

B. Stream Synchronization

When handling complex streams (like separate audio and video channels), receivers use synchronization mechanisms locally. They delay either the audio or video slightly in local buffers to ensure playback aligns perfectly (e.g., matching lip-movement in the video to the audio).

6. Application-Level Multicast and Epidemic (Gossip) Algorithms

When systems need to disseminate data to many nodes, they use specialized propagation models.

A. Application-Level Multicast (ALM)

ALM organizes nodes into an overlay network to forward data. Engineers evaluate ALM based on **Link Stress** (how many times a packet crosses the same physical link) and **Stretch** (the ratio of delay between the ALM path compared to the optimal network-level path).

B. Epidemic Propagation Models (Gossiping)

Used to synchronize state across vast networks lazily, mimicking the spread of a virus.

- **The Process:** A server S with a new update contacts other random servers. If S contacts a server that *already* has the update, S stops contacting other servers with a probability of $1/k$.
- **Efficiency:** Using a push-pull model, it only takes $O(\log N)$ rounds to disseminate updates to all N nodes in the system.

C. Data Aggregation via Gossip

Gossip algorithms are also used for decentralized computing. For example, to find the average value across a network, every node maintains a variable x_i. When two nodes gossip, they reset their variables to the average of their combined values: $x_a, x_b \rightarrow \frac{(x_a+x_b)}{2}$. Eventually, all nodes will converge on the network's global average.

D. Deleting Values (Death Certificates)

A fundamental problem in epidemic algorithms is that we cannot simply delete an old value from a server, because the system will think the server just hasn't received the data yet and will try to re-update it.

- **Solution:** Deletions are registered as a special update called a **Death Certificate**.
- **Cleanup:** To prevent death certificates from filling up system memory indefinitely, the system either runs a global garbage collection algorithm to ensure the removal is known everywhere, or it assigns the death certificate a maximum lifetime, after which it is automatically purged.

7. Importance of Message-Oriented Communication in Distributed Systems

Message-oriented communication provides several crucial benefits for the health and scale of modern systems:

- **Loose Coupling in Time and Space:** Senders and receivers do not need to be executing at the same time or know each other's exact physical locations.
- **Fault Tolerance:** If a receiving application crashes or a network partition occurs, messages safely reside in intermediate queues until the receiver recovers.
- **Heterogeneous Integration:** Message brokers enable organizations to translate protocols and data structures, seamlessly integrating disparate legacy systems.

8. Advantages of Message-Oriented Communication

Message-oriented communication, particularly persistent messaging through Message-Oriented Middleware (MOM), provides several distinct advantages that make it highly suitable for large, complex distributed systems:

- **Loose Coupling (Time and Space):** This is the most significant advantage. The sender and receiver do not need to be active at the same time (time uncoupling), nor do they need to know each other's exact network locations (space uncoupling). They only need to know which queue or topic to send to or read from.
- **Asynchronous Processing:** Senders do not have to wait (block) for the receiver to process a message. Once a message is placed in a queue, the sender can immediately resume its own tasks. This vastly improves the responsiveness and efficiency of the sending application.
- **Enhanced Fault Tolerance:** If a receiving application crashes, experiences a network partition, or undergoes maintenance, messages are not lost. They are safely buffered in persistent queues and will be delivered once the receiver comes back online.
- **Scalability and Load Balancing:** It is very easy to scale the system by adding more receivers. Multiple consumer processes can listen to the same queue, allowing the system to distribute a heavy workload across many machines effortlessly.
- **Heterogeneous Integration:** By utilizing message brokers, systems can easily integrate applications written in different programming languages, running on different operating systems, and using different data formats.

9. Disadvantages of Message-Oriented Communication

Despite its powerful capabilities, message-oriented communication introduces specific challenges and trade-offs compared to direct synchronous methods like Remote Procedure Calls (RPC):

- **Performance Overhead:** Persistent messaging is inherently slower than direct transient communication. Messages must be written to disk (for persistence), routed through intermediate nodes (brokers/routers), and acknowledged. This extra middleware layer adds latency, making it unsuitable for applications requiring real-time, microsecond-level synchronization.
- **Complexity of Architecture and Administration:** Introducing Message-Oriented Middleware means introducing a new, complex infrastructure layer. Administrators must configure and manage queues, brokers, routing rules, and storage. Debugging distributed applications also becomes much harder, as the flow of execution is asynchronous and spread across multiple independent components.
- **Lack of Immediate Feedback:** Because communication is asynchronous, the sender does not automatically know if the message was successfully processed by the receiver. If a sender needs a response or confirmation, developers must design and implement complex "reply-to" queues and correlation IDs, effectively simulating synchronous behavior over an asynchronous system.
- **Potential Single Points of Failure:** If the centralized message broker goes down and is not properly clustered or backed up, the entire communication backbone of the distributed system fails. Ensuring the high availability of the messaging infrastructure itself requires significant effort.
- **Message Ordering and Duplication Issues:** In distributed networks, guaranteeing that messages arrive in the exact order they were sent, or guaranteeing they are delivered *exactly once*, is notoriously difficult. Developers often have to write additional logic to handle duplicate messages (idempotency) or to re-sort out-of-order packets.

Summary

Communication is the foundational heart of any distributed system. Because these systems lack shared memory, they rely entirely on the exchange of information across networks. To overcome the rigid, synchronous, and tightly coupled limitations of traditional Remote Procedure Calls (RPCs), **message-oriented communication** evolved to offer greater flexibility.

Through the use of layered middleware, communication in distributed systems takes several distinct forms to meet different application needs:

- **Transient and Persistent Messaging:** Ranges from high-speed, direct message-passing (like MPI) to robust Message-Oriented Middleware (MOM) that uses intermediate queues and message brokers to ensure delivery even when systems are offline.
- **Stream-Oriented Communication:** Manages continuous data (like audio and video) by strictly controlling network transmission rates (Quality of Service) and synchronizing streams at the receiver end.
- **Multicast and Epidemic Algorithms:** Enables efficient data dissemination and state synchronization across vast networks using overlay trees and decentralized gossip propagation.

The Trade-off: Ultimately, message-oriented communication provides unparalleled advantages in making systems highly scalable, fault-tolerant, and loosely coupled in both time and space. It allows entirely different and heterogeneous systems to integrate seamlessly. However, architects must balance these benefits against the inherent disadvantages, which include increased architectural complexity, performance overhead from persistent storage, and the challenges of managing asynchronous execution without immediate feedback.

Understanding these communication models, along with their respective middleware mechanisms, is fundamental to designing and maintaining robust, modern distributed architectures.

Unit 4 –Communication Understanding Remote Procedure Calls (RPC)

LEE CARLO F. SIMON

Content

- What is RPC?
- How RPC Works?
- Importance of RPC in Distributed Systems
- Types of RPC
- Advantages and Challenges of RPC
- Real-World Use Cases of RPC

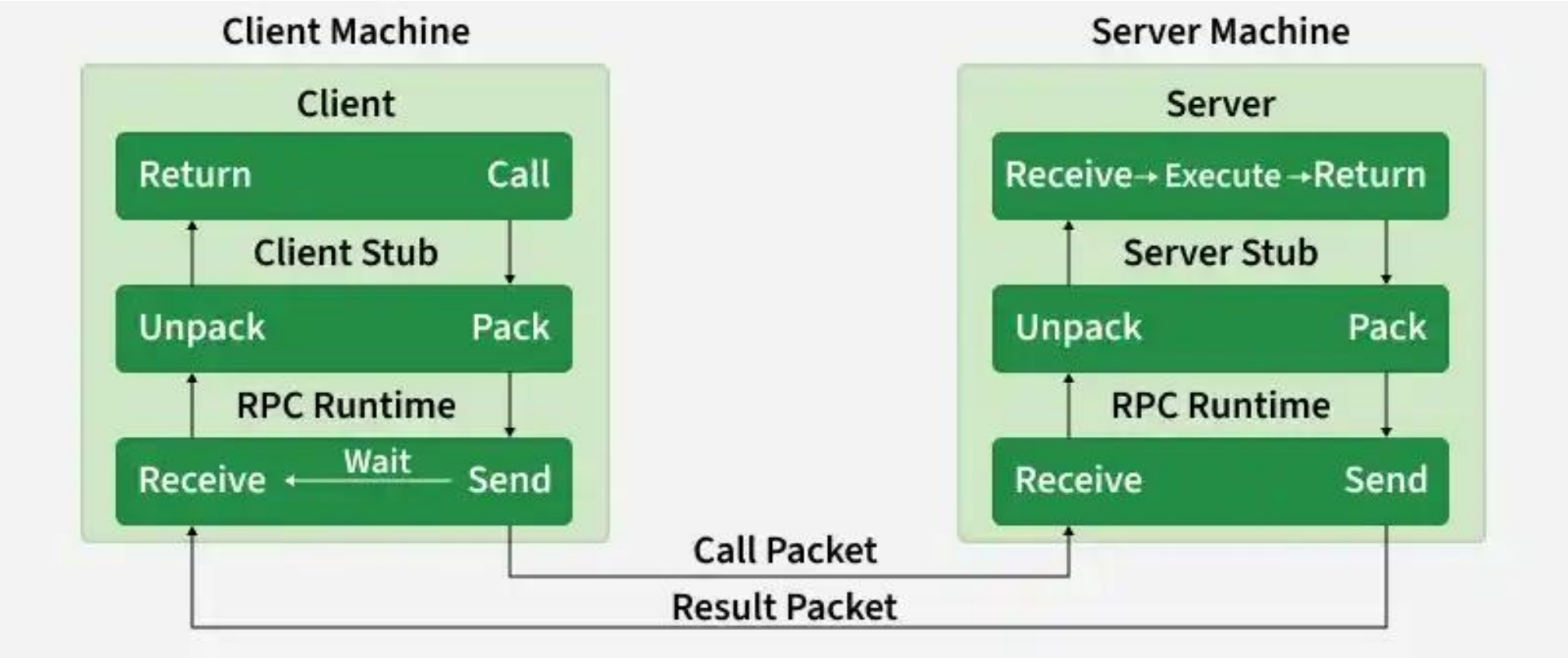
Remote Procedure Calls (RPC)

In distributed systems, nodes (or computers) often need to collaborate to perform tasks. But how does one computer tell another to execute a function or operation? This is where Remote Procedure Calls (RPC) come into play.

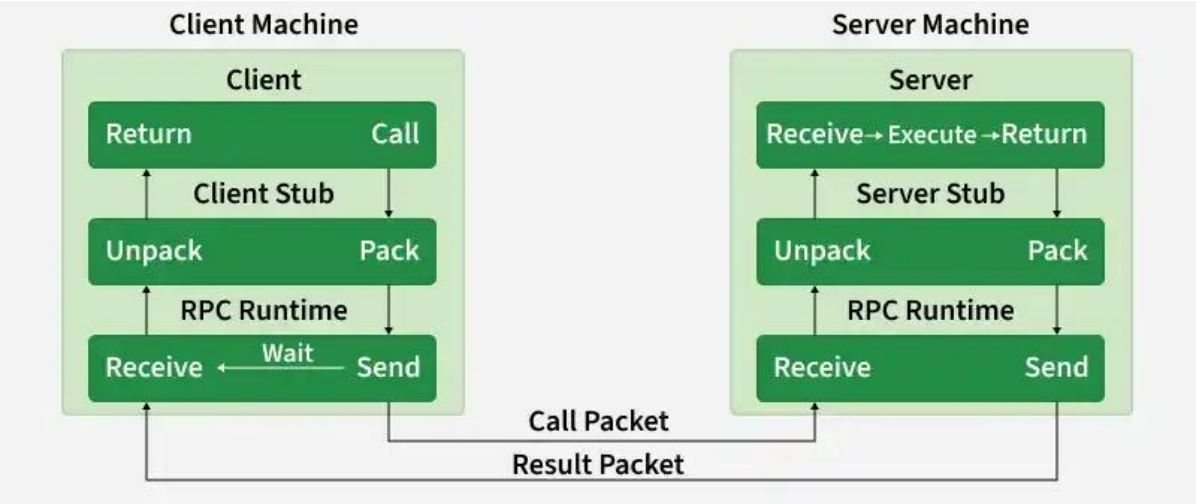
What is RPC?

RPC stands for Remote Procedure Call. It's a communication method that allows a program to execute a function or procedure on another computer (or node) as if it were a local function call.

How RPC Works?



How RPC Works?

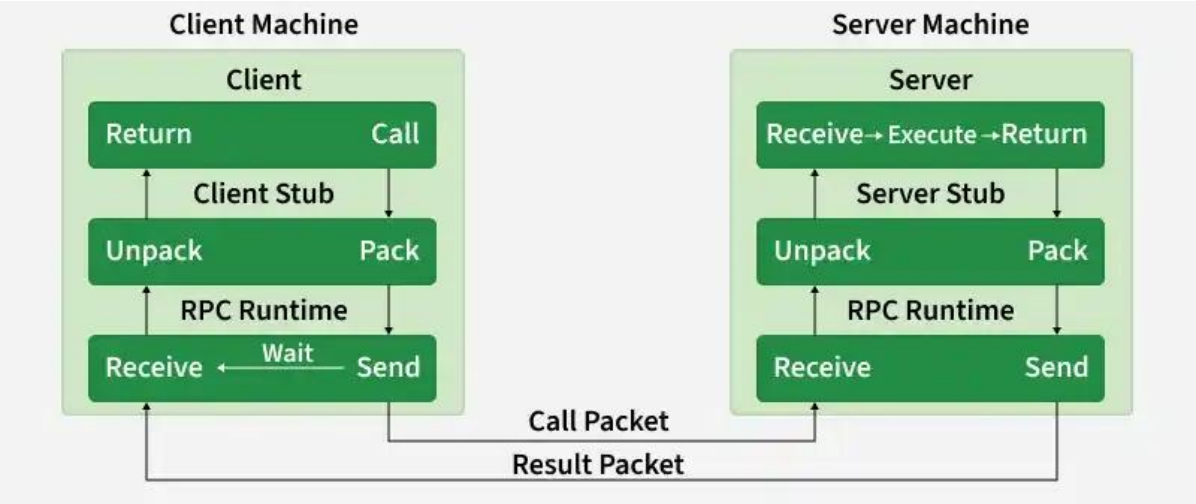


Client: The client process initiates RPC. The client makes a standard call, which triggers a correlated procedure in the client stub.

Client Stub: Stubs are used by RPC to achieve semantic transparency. The client calls the client stub. Client stub does the following tasks:

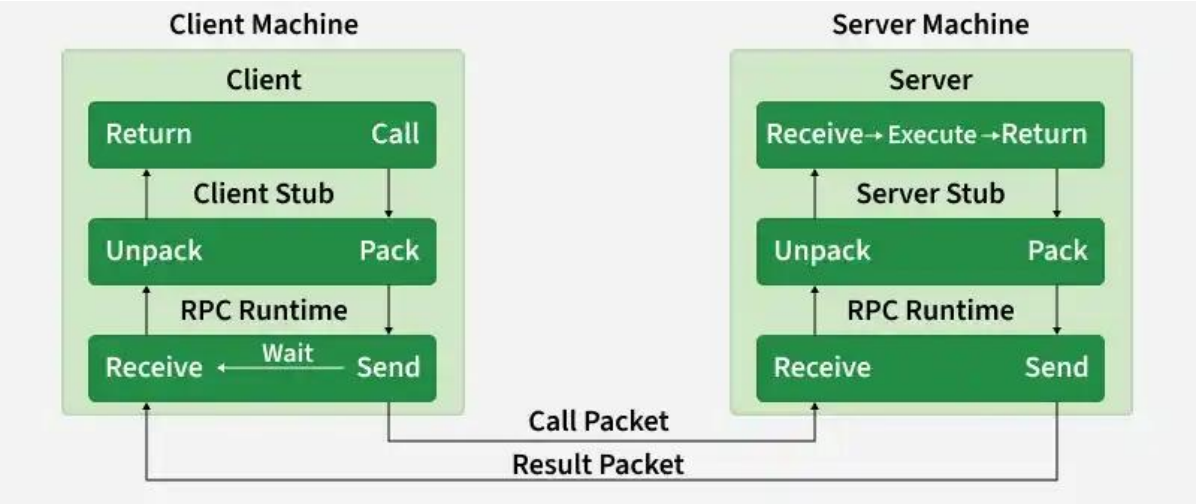
- The first task performed by the client stub is when it receives a request from a client, it packs(marshals) the parameters and required specifications of the remote/target procedure in a message.
- The second task performed by the client stub is upon receiving the result values after execution, it unpacks (unmarshalls) those results and sends them to the Client.

How RPC Works?



RPC Runtime: The RPC runtime is responsible for message transmission between the client and the server over the network. Retransmission, acknowledgement, routing, and encryption are all tasks it performs. On the client-side, it receives the result values in a message from the server-side and then sends them to the client stub. On the server-side, the RPC Runtime receives the same message from the server stub and forwards it to the client machine. It also accepts and forwards client machine call request messages to the server stub.

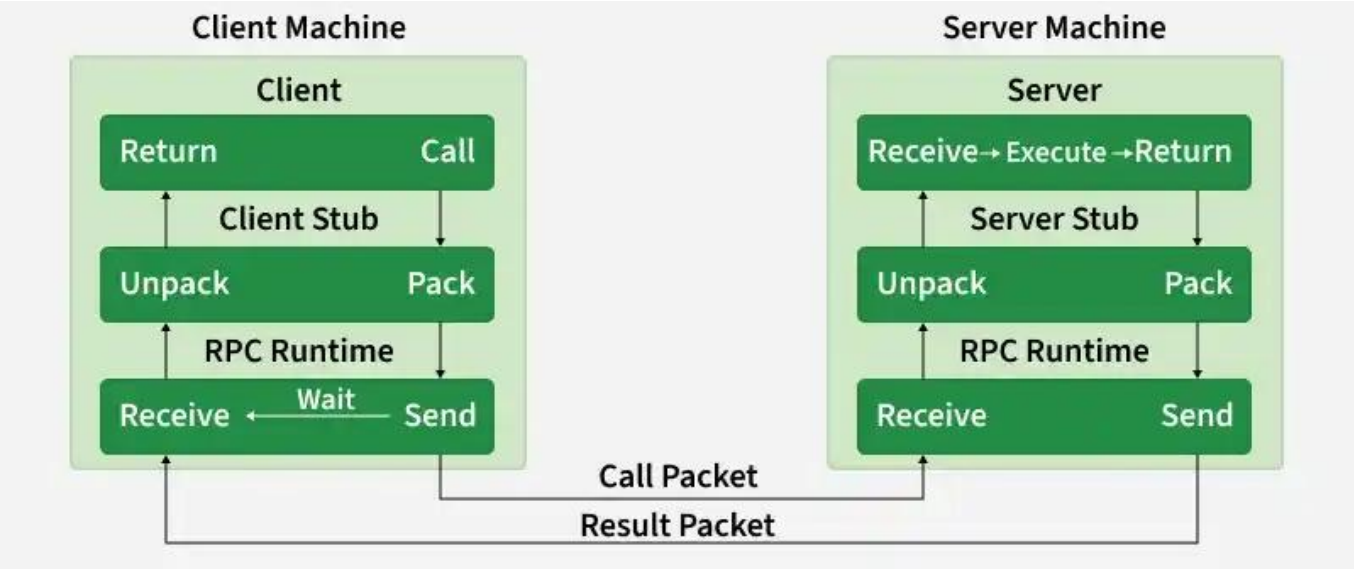
How RPC Works?



Server Stub: Server stub does the following tasks:

- The first task performed by server stub is that it unpacks(unmarshalled) the call request message which is received from the local RPC Runtime and makes a regular call to invoke the required procedure in the server.
- The second task performed by the server stub is that when it receives the server's procedure execution result, it packs it into a message and asks the local RPC Runtime to transmit it to the client stub, where it is unpacked.

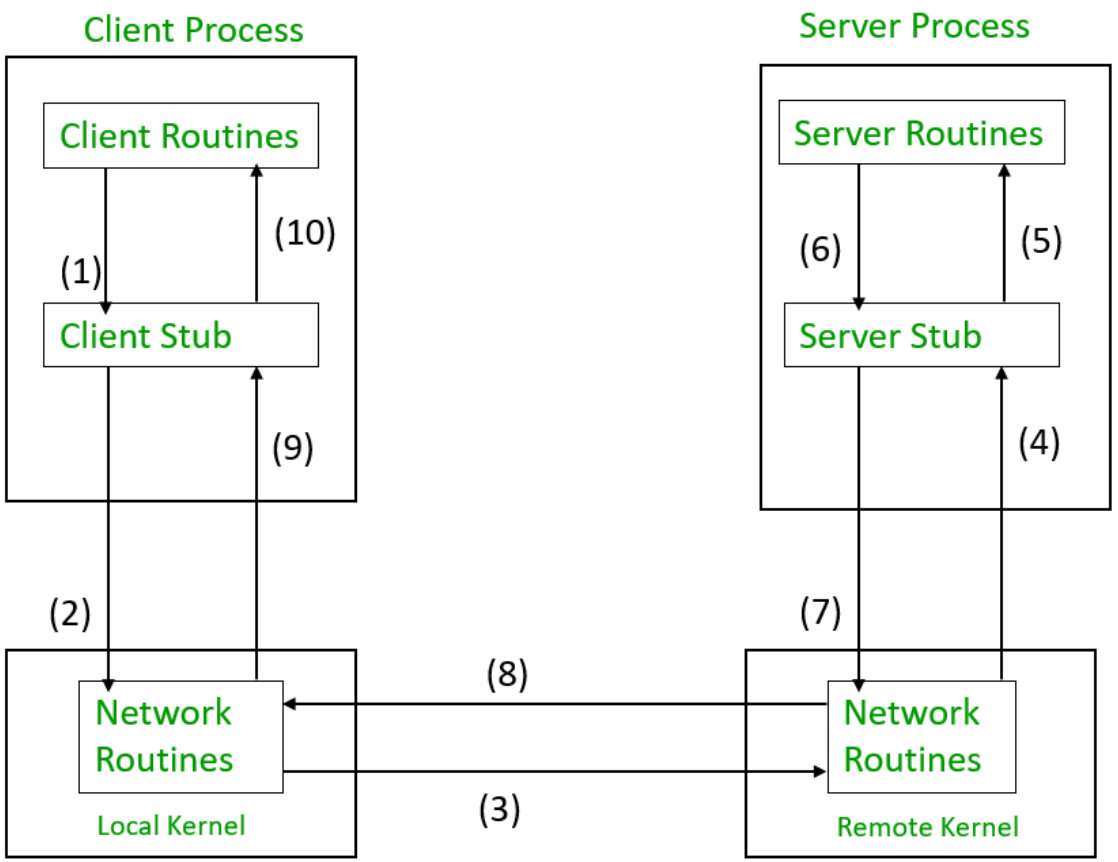
How RPC Works?



Server: After receiving a call request from the client machine, the server stub passes it to the server. The server performs the execution of the required procedure, and it finally returns the result to the server stub so that it can be passed to the client machine using the local RPC Runtime.

How RPC Works?

1. The client procedure initiates the remote procedure call by calling the client stub in the normal way.
2. The client stub builds a message that contains the procedure name and its parameters and then calls the local operating system (OS).
3. The client's OS sends the message over the network to the remote OS.
4. The remote OS receives the message and passes it to the server stub.
5. The server stub unpacks the parameters from the message and then calls the actual server procedure.
6. The server processes the request and returns the result to the server stub.
7. The server stub packs the result into a message and then calls the local OS.
8. The server's OS sends the message over the network to the client's OS.
9. The client's OS receives the message and then gives it to the client stub.
10. The client stub unpacks the result from the message and then returns it to the client procedure.



How RPC Works?

Think of it like ordering food at a restaurant:

You (the client) tell the waiter (the RPC mechanism) what you want.

The waiter communicates with the kitchen (the server) to prepare the food.

The kitchen (server) completes the order and gives it to the waiter.

The waiter delivers the food back to you.

In this analogy:

You = The client node
The waiter = RPC framework
The kitchen = The server node

Importance of RPC in Distributed Systems

Simplified Communication

- Abstraction of complexity: Developers call remote services as if they were local methods.
- Consistent interface: Uniform way of invoking services across the system.

Enhanced Modularity and Reusability

- Decoupling: System components interact without being tightly coupled.
- Reusability: Remote services can be reused across multiple applications.

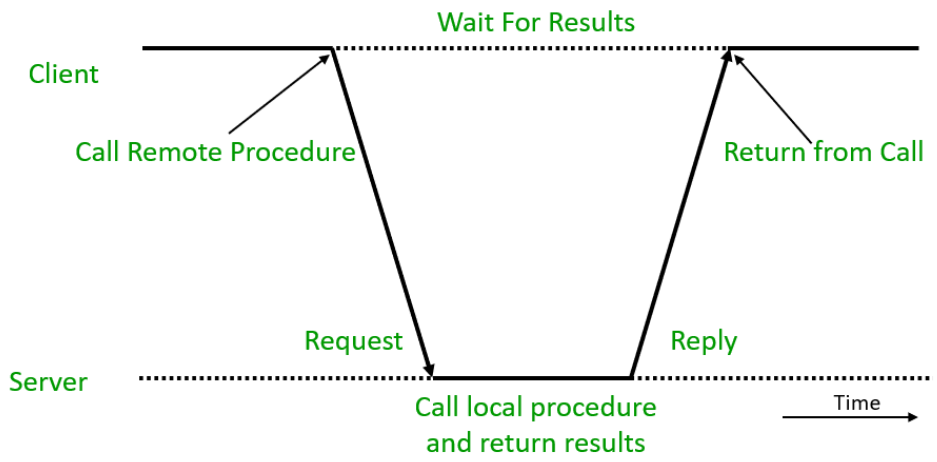
Facilitates Distributed Computing

- Inter-Process Communication (IPC): Enables different processes on different machines to collaborate.
- Resource Sharing: Allows sharing of resources like databases, computing power, or specialized services.

Types of Remote Procedure Calls (RPC)

1. Synchronous RPC

In a synchronous RPC, the client sends a request to the server and waits for the response before continuing execution.



Characteristics:

- Blocks the client until the server processes the request and sends the response.
- Simplifies programming as it behaves like a local function call.
- May introduce latency due to waiting for network communication and server processing.

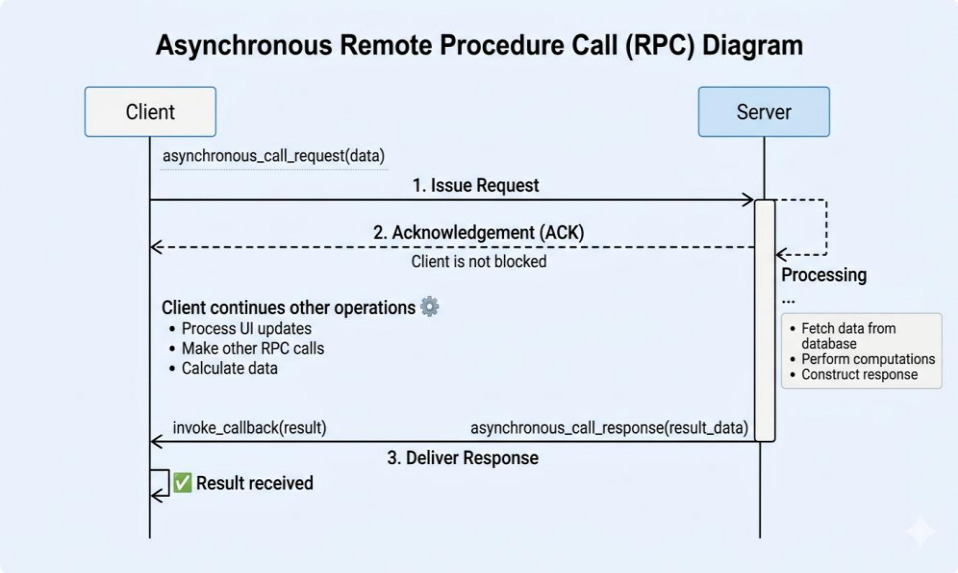
Pros: Easy to implement; predictable control flow; similar to standard programming practices.

Cons: Can lead to performance bottlenecks; prone to "freezing" the UI if used on a main thread; highly sensitive to network latency.

Types of Remote Procedure Calls (RPC)

2. Asynchronous RPC

A communication pattern where a client initiates a request to a remote server and then immediately continues its execution without waiting for the server to respond.



Characteristics:

- Non-blocking; allows the client to perform other tasks while waiting for the response.
- Adds complexity as the client must handle responses asynchronously.
- Reduces idle time and improves overall efficiency in systems with high latency.

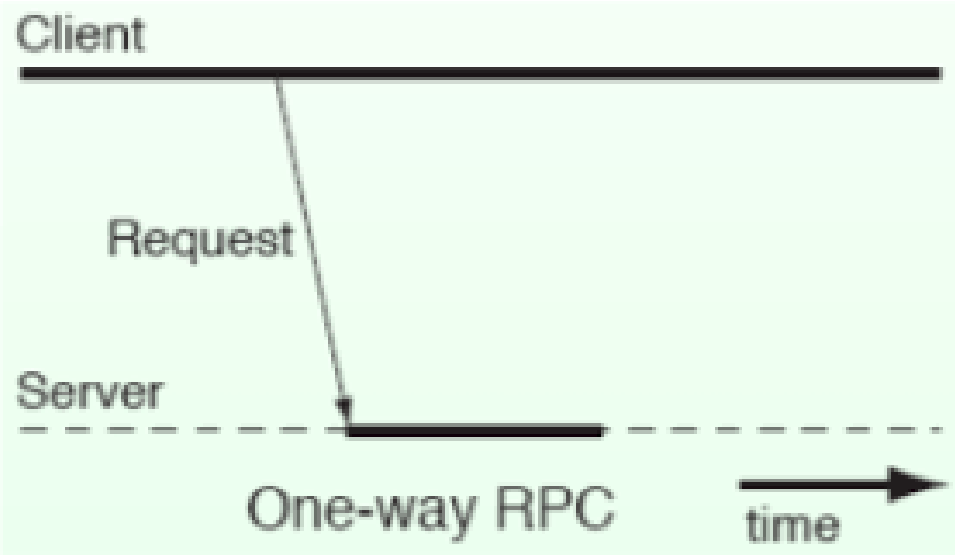
Pros: include higher responsiveness and parallelism as the client is not blocked.

Cons: involve increased system complexity and more difficult error handling compared to the straightforward, synchronous model.

Types of Remote Procedure Calls (RPC)

3. One-Way RPC

a "fire-and-forget" communication pattern where the client sends a request to the server but does not expect or wait for any response, not even an acknowledgement (ACK).



Characteristics:

- No Feedback Loop: The client has no way of knowing if the server received the message or if the task was completed.
- Maximum Performance: Since there is zero waiting time for a round-trip response, it is the fastest RPC method for the client.
- Unreliable: It is generally used in scenarios where losing a few messages is acceptable (best-effort delivery).

.

Advantages and Challenges of RPC

1. Simplicity

- Developers can call remote functions without worrying about network complexities.

2. Abstraction

- The network communication details (like serialization and deserialization) are hidden from the developer.

3. Efficiency

- Frameworks like gRPC provide high-performance communication with low latency.

4. Cross-Language Support

- Many RPC frameworks allow communication between services written in different programming languages.

Advantages and Challenges of RPC

1. Network Failures

- Since RPC relies on a network, failures can lead to delays or errors.

2. Latency

- Remote calls are slower than local calls due to network overhead.

3. Complexity in Debugging

- Debugging distributed systems with RPC can be more challenging than debugging monolithic systems.

4. Compatibility Issues

- Different versions of services may cause incompatibilities during communication.

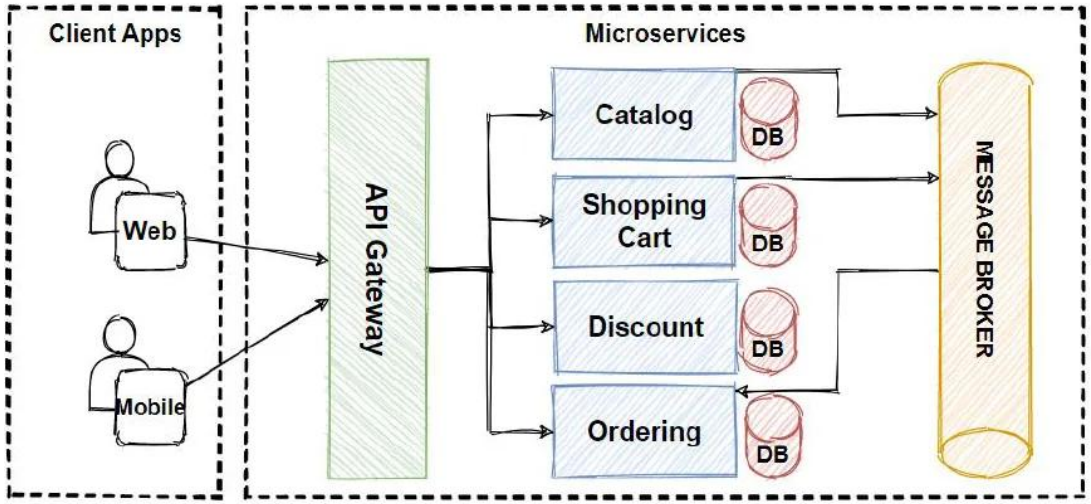
Real-World Use Cases of RPC

1. Microservices Architecture

- In a microservices system, different services handle different parts of an application (e.g., payment, user management, inventory).

Example:

- The **Order Service** calls the **Payment Service** via RPC to process a transaction.
- The **Inventory Service** updates stock levels after the transaction is successful.



Real-World Use Cases of RPC

2. Distributed Databases

- Distributed databases like Cassandra or Etcd use RPC to synchronize data between nodes.

Example: When a client writes data to one node, the system uses RPC to replicate the data across other nodes.

.

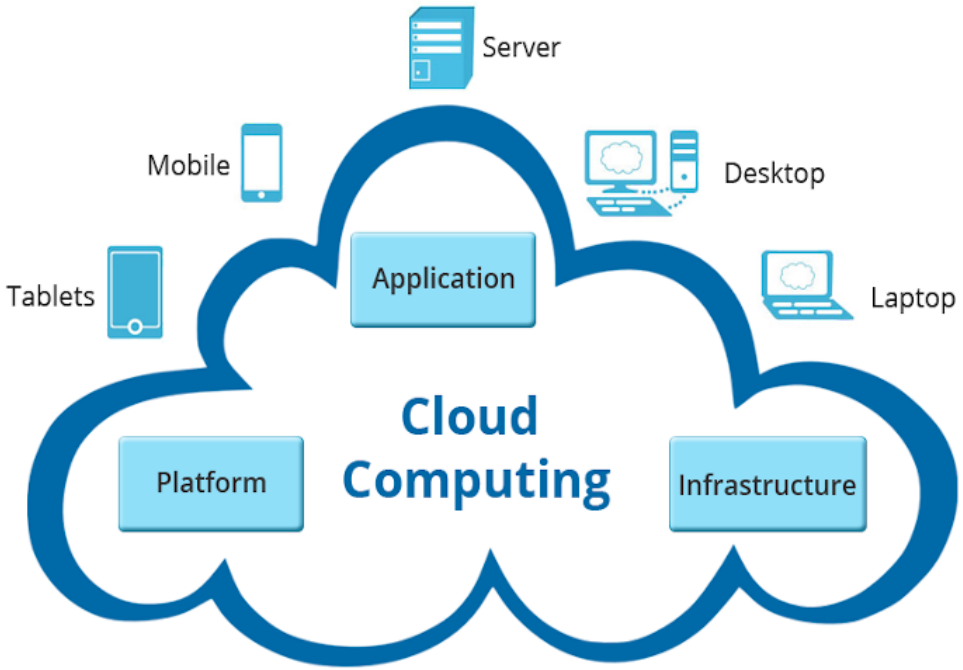
3. Authentication Systems

- An authentication service might use RPC to verify login credentials by calling a user database service.

Example: Logging into a social media app sends an RPC to check your username and password against the database.

.

Real-World Use Cases of RPC



4. Cloud-Based Applications

- RPC is the backbone of many cloud services where APIs allow clients to interact with remote servers.

Example: Video streaming platforms like Netflix use RPC to fetch user preferences and recommend content.

.

Next Topic

Unit 4. Communication

Message-Oriented Middleware (MOM)

Unit 4 - Communication

LEE CARLO F. SIMON

Content

- Introduction
- The Strategic Importance of Communication
- Layered Protocols and the OSI Model

Introduction

In a distributed system, there is no shared memory. If Node A needs to tell Node B something, it must package that information into a message and send it over a network. This process is far more complex than a local function call because messages can be lost, delayed, or received out of order. Communication in distributed systems is governed by **Protocols**—sets of rules that ensure both sides understand the data being exchanged. This unit explores how we hide these complexities using **Middleware** and the different ways processes synchronize their actions.

The Strategic Importance of Communication

Communication is the "lifeblood" of a distributed system. Because a distributed system is defined by its lack of shared memory—meaning no single processor can directly see the data stored in another processor's RAM—every single action, from logging in to updating a database, depends entirely on how nodes talk to one another.

The Strategic Importance of Communication

1. **Collaboration Between Autonomous Elements**

A distributed system consists of independent nodes. Without communication, these nodes are just isolated computers. Communication allows these separate "autonomous elements" to synchronize their states, share tasks, and work toward a common goal, creating the illusion of a Single Coherent System.

Example: **Google Docs.**

The Strategic Importance of Communication

- **2. Achieving Distribution Transparency**

One of the core goals of a distributed system is to hide where resources are located. Communication protocols and middleware make this possible.

Example: When you access a cloud file, communication happens in the background to find which server has that file and bring it to you. You don't see the "messages" being sent; you only see your file. This is called **Location Transparency**.

Example: **An ATM (Automated Teller Machine)..**

The Strategic Importance of Communication

3. Overcoming the "No Shared Memory" Barrier

In a standard PC, the CPU and GPU talk through a shared bus (physical wires on the motherboard). In a distributed system, the "bus" is the Network. Communication mechanisms like **Remote Procedure Calls (RPC)** act as the bridge that allows a program on one machine to execute logic on another.

Example: **Online Multiplayer Games**

The Strategic Importance of Communication

4. Maintaining Consistency and State

If you change your password on your smartphone, that change must be communicated to the central server and then to your laptop. Without reliable communication, different nodes would have different information, leading to a "split-brain" scenario where the system becomes unreliable.

Example: **Facebook/Social Media Notifications**

The Strategic Importance of Communication

5. Handling Scalability and Load Balancing

Communication allows a system to grow. When one server becomes too busy, it communicates with a "Load Balancer" to redirect new requests to an idle server. This **Horizontal Scaling** is only possible if nodes can constantly exchange status updates about their current workload.

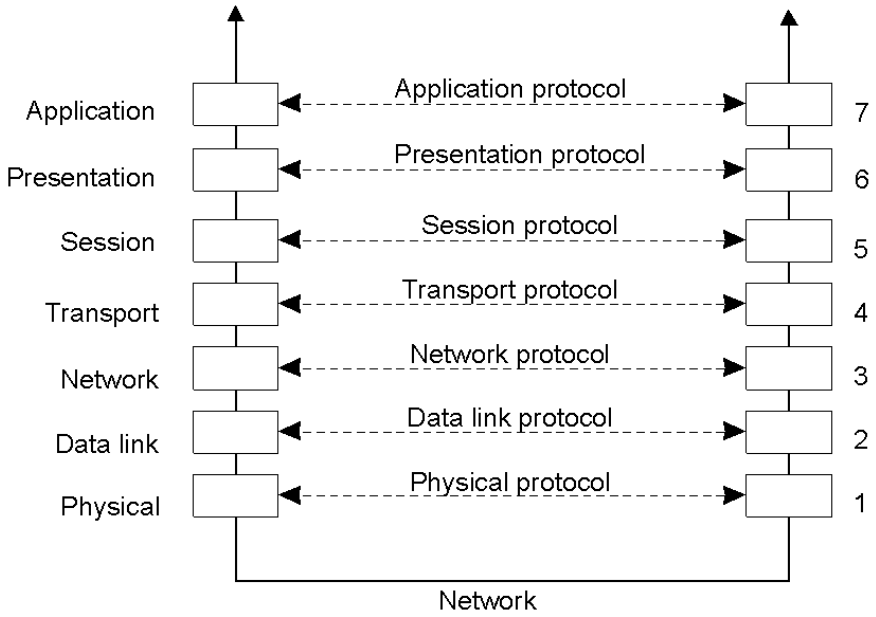
Example: **Netflix Streaming**

Layered Protocols and the OSI Model

1. The Concept of Abstraction

The primary goal of layering is **abstraction**. For example, when you send an email, the application doesn't need to know if you are using a Wi-Fi connection or an Ethernet cable. It simply hands the data to the layer below. Each layer communicates with its "peer" on the other machine using a set of rules called a **Protocol**.

Layered Protocols and the OSI Model



2. The OSI Reference Model

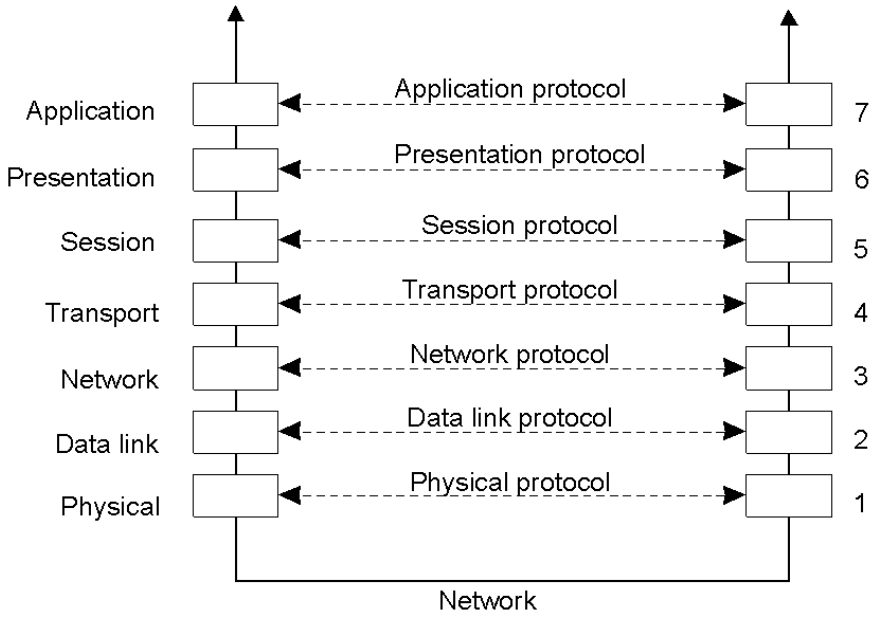
The International Standards Organization (ISO) developed the **Open Systems Interconnection (OSI)** model, which consists of seven layers. In distributed systems, we generally group these into three functional categories:

- **The Low-Level Layers (Layers 1-3):** * **Physical:** Handles bit-stream transmission over physical media (cables, radio waves).

  Data Link: Detects and corrects errors that occur in the physical layer.

  Network: Responsible for routing. It decides which path a "packet" should take to reach its destination (e.g., IP addresses).

Layered Protocols and the OSI Model



- **The Transport Layer (Layer 4):**

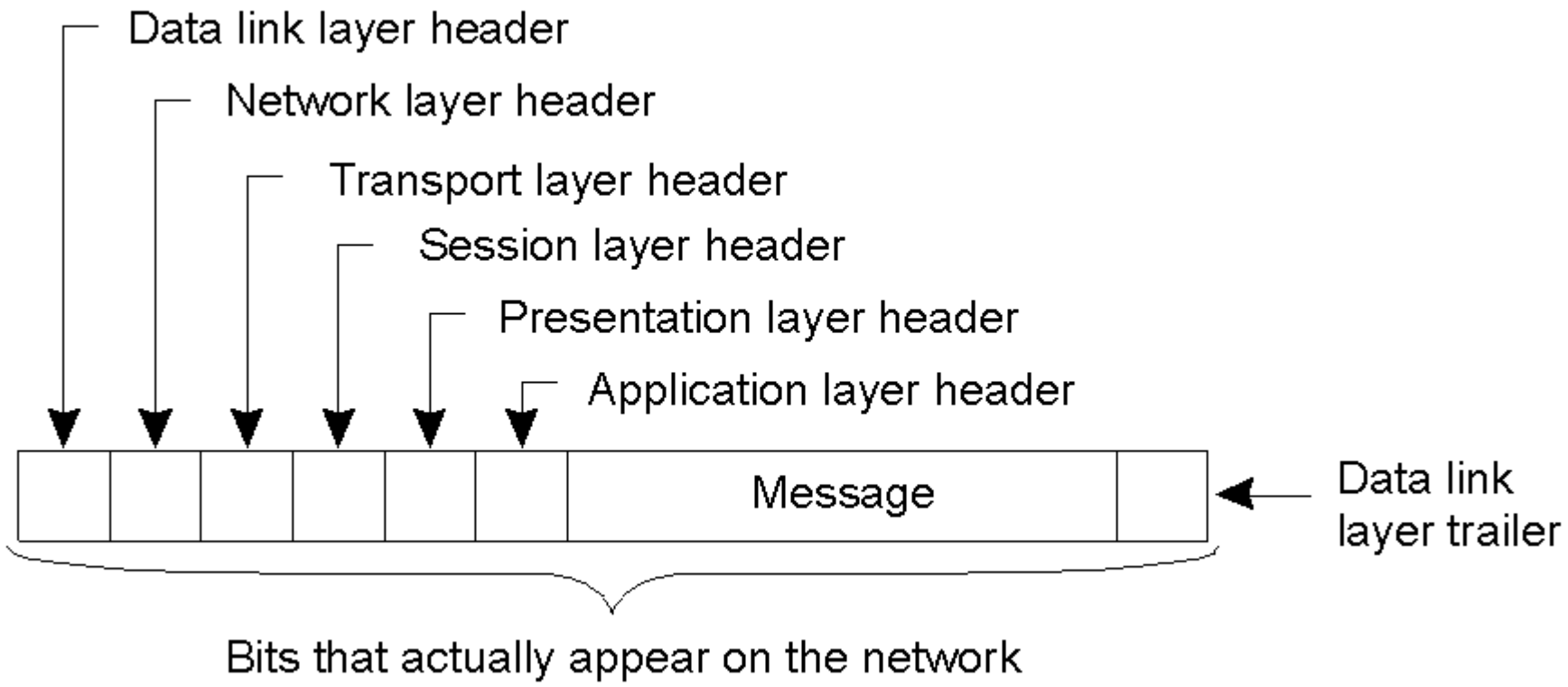
  This is the "bridge" between the network and the application. It ensures that messages are delivered in the correct order and without loss. Protocols like **TCP** (Transmission Control Protocol) live here.

- **The Upper Layers (Layers 5-7):**

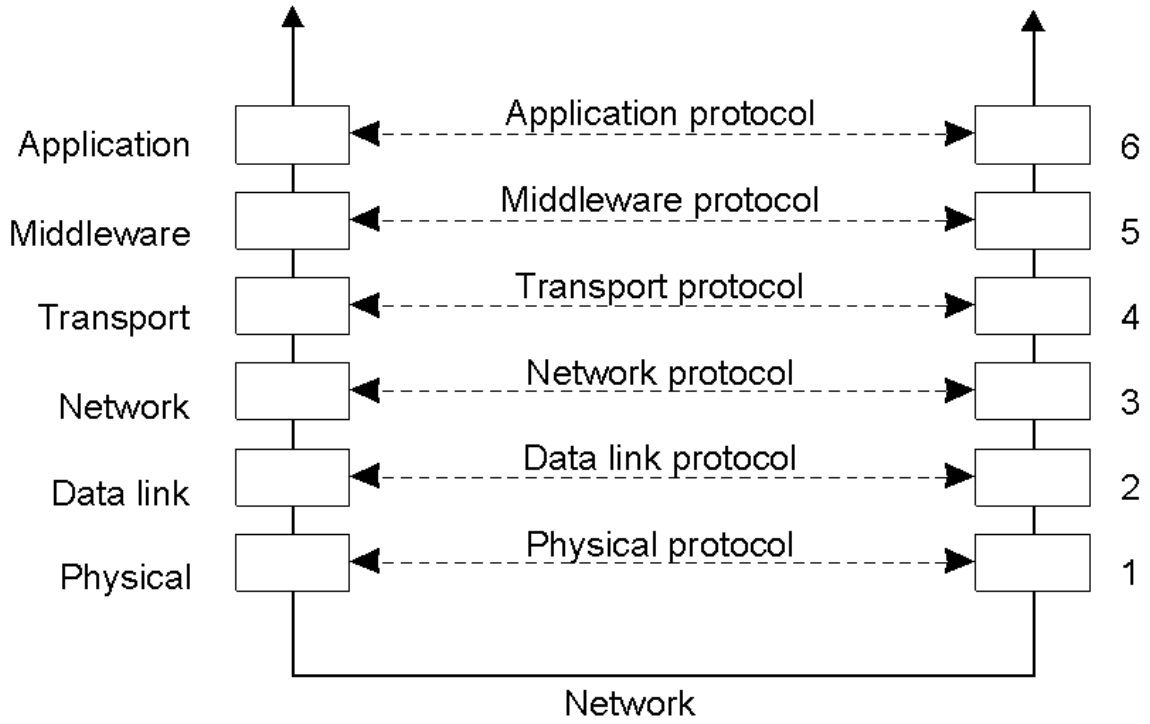
  Session & Presentation: Handle data encryption and formatting (e.g., converting a Mac's data format to a Windows format).

  Application: Where the actual distributed applications reside (e.g., HTTP for the web, FTP for files).

Layered Protocols and the OSI Model



Layered Protocols and the OSI Model



3. Middleware and the "Gap"

In modern distributed systems, the standard OSI model isn't enough. There is often a "gap" between the Transport layer and the Application layer. This is where **Middleware** resides.

Middleware acts as an "extra layer" that provides high-level communication services, such as:

- **Remote Procedure Calls (RPC):** Allowing a program to call a function on a different machine as if it were local.
- **Naming Services:** Looking up a server by name rather than a confusing IP address.
- **Security:** Automatically encrypting all communication between nodes.

Next Topic

Unit 3. Communication

Remote Procedure Call (RPC)